More Backtracking Examples

Prof. Timothy Sun

Subset Sum

TSP is an example of an ***NP-hard*** problem. Another is SUBSET-SUM:

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

Subset Sum

TSP is an example of an ***NP-hard*** problem. Another is SUBSET-SUM:

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

S = {6, 12, 8, 18, 3, 19}, ***T*** = 39

Subset Sum

Target: 39

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

S = {6, 12, 8, 18, 3, 19}, ***T*** = 39

YES, 8+12+19 = 39 or 3+6+12+18 = 39

Bin-packing, knapsack, partition, etc.

Subset Sum is one variant of a general class of “packing” problems.

- Bin-packing: fewest number of trips
- Knapsack: one trip, maximize value
- Subset sum: exactly at capacity
- and so on…

Bin-packing, knapsack, partition, etc.

Subset Sum is one variant of a general class of “packing” problems.

- Bin-packing: fewest number of trips
- Knapsack: one trip, maximize value
- Subset sum: exactly at capacity
- and so on…

Most interesting variations are NP-hard!

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

S = {6, 12, 8, 18, 3, 19}, ***T*** = 50

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

S = {6, 12, 8, 18, 3, 19}, ***T*** = 50

NO SOLUTION

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

Goal: design an algorithm for solving Subset Sum

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

Goal: design an algorithm for solving Subset Sum

Brute-force using backtracking

Subset Sum

Given a set of positive integers ***S*** and a target ***T***, is there a subset of ***S*** that adds up to ***T***?

Goal: design an algorithm for solving Subset Sum

Brute-force using backtracking: effectively try every subset

The set of subsets

If $\boldsymbol{S}$ is a set of $\boldsymbol{n}$ elements, how many subsets are there? (including $\boldsymbol{S}$ and $\varnothing$)

The set of subsets

If $\boldsymbol{S}$ is a set of $\boldsymbol{n}$ elements, how many subsets are there? (including $\boldsymbol{S}$ and $\varnothing$)

Ans: 2^n

The set of subsets

If S is a set of n elements, how many subsets are there? (including S and $\varnothing$)

Ans: 2^n

Proof: we can pick a subset $X \subseteq S$ in the following way:

- For each element, either:
  - Include it
  - Exclude it

Two possible choices for each element, so there are 2^n elements.

Base cases

Two parameters:

- ***S***, a set of positive numbers
- ***T***, a target value

Base cases

Two parameters:

- ***S***, a set of positive numbers
- ***T***, a target value

Base cases: ***S*** is empty, and/or ***T*** ≤ 0.

Base cases

When ***S*** is empty, what values of ***T*** can it add up to?

Base cases

When ***S*** is empty, what values of ***T*** can it add up to?

Ans: only subset is itself, which has sum 0.

$$
\begin{aligned}
&\textsc{Sum}(A[1,\ldots,n]):\\
&\qquad sum := \boxed{0}\\
&\qquad \textbf{for } i = 1,\ldots,n\\
&\qquad\qquad sum := sum + A[i]\\
&\qquad \textbf{return } sum
\end{aligned}
$$

Base cases

When $T \leq 0$, which sets S (of positive integers) have a subset that adds up to T?

Base cases

When $T \leq 0$, which sets S (of positive integers) have a subset that adds up to T?

- If $T < 0$, then nothing
- If $T = 0$, then every set S has the empty set as a subset.

Base cases

Coding up the base cases...

```
SubsetSum(S, T):
    if T = 0
        return True
    else if T < 0 or S = ∅
        return False
    else
        // some stuff here
```

Enumerating subsets

Let ***e*** be any element of ***S***. Every subset ***X*** of ***S*** either:

- has ***e***
- doesn't have ***e***

Enumerating subsets

Let ***e*** be any element of ***S***. Every subset ***X*** of ***S*** that sums to ***T*** either:

- has ***e*** — ***X*** - {***e***} sums to ***T*** - ***e***
- doesn't have ***e*** — ***X*** - {***e***} = ***X*** sums to ***T***

Enumerating subsets

Let ***e*** be any element of ***S***. Every subset ***X*** of ***S*** that sums to ***T*** either:

- has ***e*** — ***X*** - {***e***} sums to ***T*** - ***e***
- doesn't have ***e*** — ***X*** - {***e***} = ***X*** sums to ***T***

Algorithm: recurse on ***S*** - {***e***} with ***T*** and ***T*** - ***e*** as the new targets.

Enumerating subsets

```
SubsetSum(S, T):
    if T = 0
        return True
    else if T < 0 or S = ∅
        return False
    else
        pick e ∈ S
        include := SubsetSum(S − {e}, T − e)
        exclude := SubsetSum(S − {e}, T)
        return include or exclude
```

Enumerating subsets

If S is given as an array $S[1, \ldots, n]$, how do we quickly find e and $S - \{e\}$?

Enumerating subsets

If S is given as an array $S[1, \ldots, n]$, how do we quickly find e and $S - \{e\}$?

- $e = S[n]$
- $S - \{e\} = S[1, \ldots, n-1]$

Enumerating subsets

```
SubsetSum(S[1,...,n], T):
    if T = 0
        return True
    else if T < 0 or n = 0
        return False
    else
        e := S[n]
        include := SubsetSum(S[1,...,n-1], T-e)
        exclude := SubsetSum(S[1,...,n-1], T)
        return include or exclude
```

Enumerating subsets

```
SubsetSum(S[1,...,n], i, t):
    if t = 0
        return True
    else if t < 0 or i = 0
        return False
    else
        e := S[i]
        include := SubsetSum(S, i − 1, t − e)
        exclude := SubsetSum(S, i − 1, t)
        return include or exclude
```

Enumerating subsets

```
SubsetSum(S[1,...,n], i, t):
    if t = 0
        return True
    else if t < 0 or i = 0
        return False
    else
        e := S[i]
        include := SubsetSum(S, i − 1, t − e)
        exclude := SubsetSum(S, i − 1, t)
        return include or exclude
```

Initial call: $\textsc{SubsetSum}(S, n, T)$

Enumerating subsets

$\textsc{SubsetSum}(\boxed{S[1,\ldots,n], i,} t)$:

if $t = 0$

return $\textsc{True}$

else if $t < 0$ or $\boxed{i = 0}$

return $\textsc{False}$

else

$e := \boxed{S[i]}$

$\mathit{include} := \textsc{SubsetSum}(\boxed{S, i-1,} t - e)$

$\mathit{exclude} := \textsc{SubsetSum}(\boxed{S, i-1,} t)$

return *include* or *exclude*

Initial call: $\textsc{SubsetSum}(S, n, T)$

This solution ***indexes*** the subproblems. (We'll see why later)

Enumerating subsets

```
SubsetSum(S[1,...,n], i, t):
    if t = 0
        return True
    else if t < 0 or i = 0
        return False
    else
        e := S[i]
        include := SubsetSum(S, i − 1, t − e)
        exclude := SubsetSum(S, i − 1, t)
        return include or exclude
```

$$T(n) = 2\,T(n-1) + O(1) = \Theta(2^n)$$

Backtracking guidelines

1. Identify recursive structure and subproblems (recurrence)
2. Solve the base cases
3. Try all first choices ("take a nibble"), and let recursion take care of the rest.

```
SubsetSum(S[1,...,n], i, t):
    if t = 0
        return True
    else if t < 0 or i = 0
        return False
    else
        e := S[i]
        include := SubsetSum(S, i − 1, t − e)
        exclude := SubsetSum(S, i − 1, t)
        return include or exclude
```

Variations

This code only checks if there exists a solution. How can we modify it to output:

- Output the actual solution?
- The number of solutions?
- The “best” solution?

Variations

This code only checks if there exists a solution. How can we modify it to output:

- Output the actual solution?
- **The number of solutions?**
- The “best” solution?

Pure functions are easy to modify!

Counting the number of solutions

```
SubsetSum(S[1,...,n], i, t):
    if t = 0
        return TRUE
    else if t < 0 or i = 0
        return FALSE
    else
        e := S[i]
        include := SubsetSum(S, i − 1, t − e)
        exclude := SubsetSum(S, i − 1, t)
        return include or exclude
```

Counting the number of solutions

$\textsc{CountSubsets}(S[1,\ldots,n], i, t)$:

if $t = 0$

return $\boxed{1}$

else if $t < 0$ or $i = 0$

return $\boxed{0}$

else

$e := S[i]$

$\mathit{include} := \textsc{CountSubsets}(S, i-1, t-e)$

$\mathit{exclude} := \textsc{CountSubsets}(S, i-1, t)$

return $\boxed{\mathit{include} + \mathit{exclude}}$

Wood cutting

Given a beam of wood ***n*** units long and selling prices ***p***[***i***] for a piece ***i*** units long, what's the most money you can make by chopping up the cord?

Wood cutting

length	1	2	3	4	5	6	7	8	9	10
price	1	5	8	9	10	17	17	20	24	30

Wood cutting

length	1	2	3	4	5	6	7	8	9	10
price	1	5	8	9	10	17	17	20	24	30

$10

Wood cutting

length	1	2	3	4	5	6	7	8	9	10
price	1	5	8	9	10	17	17	20	24	30

$\$10$

$\$5 + \$8 = \$13$

Wood cutting

Sequence of choices: cuts in the piece of wood

Wood cutting

Sequence of choices: cuts in the piece of wood

Base case: length 0, no money

Wood cutting

Sequence of choices: cuts in the piece of wood

Base case: length 0, no money

First choice/nibble: cut out the leftmost piece. How should we cut it?

Wood cutting

Sequence of choices: cuts in the piece of wood

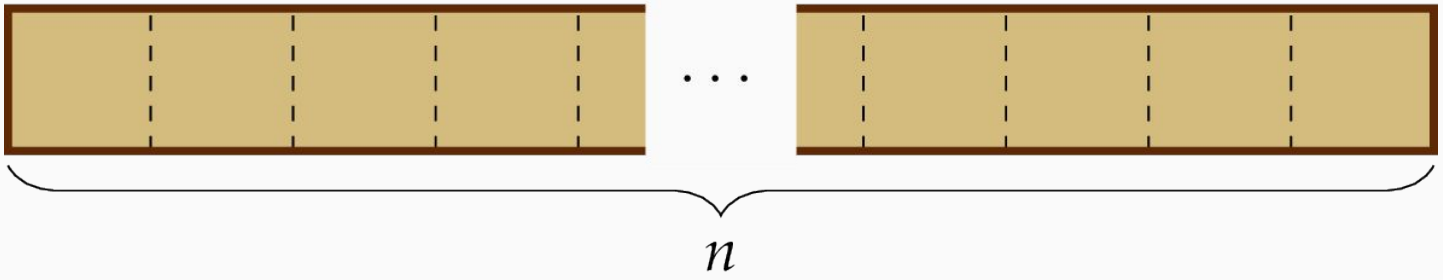
Base case: length 0, no money

First choice/nibble: cut out the leftmost piece. How should we cut it?

Try each possibility and see which one's best.

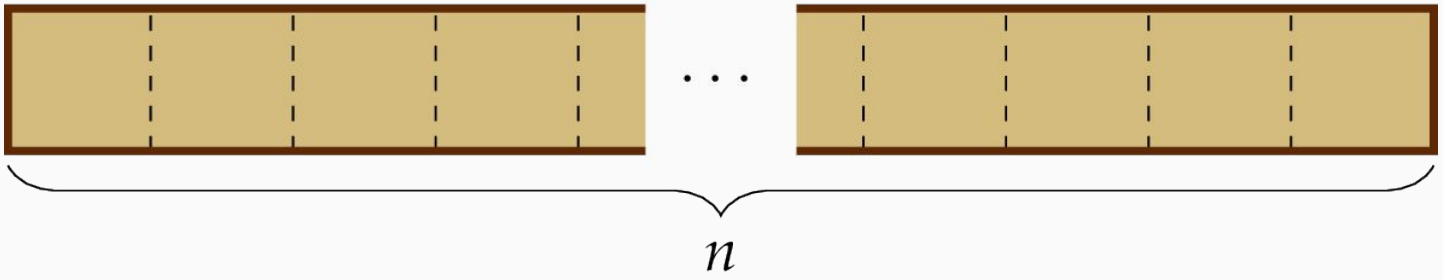
Wood cutting

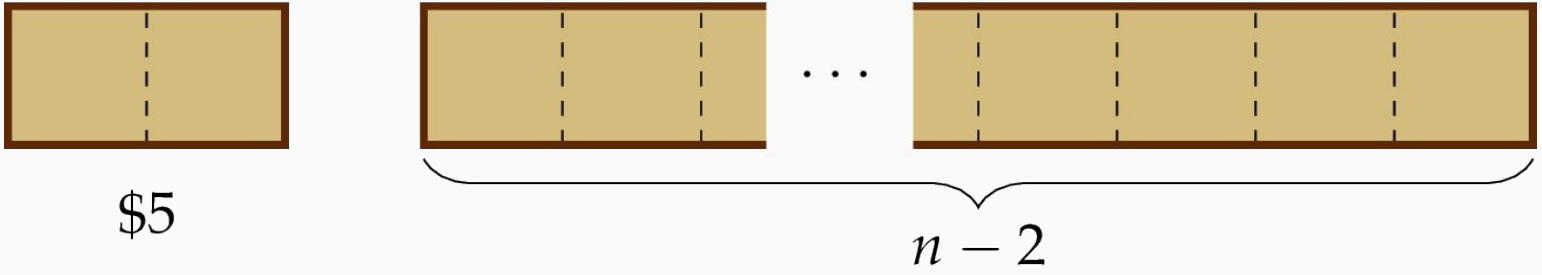
length	1	2	3	4	5	6	7	8	9	10
price	1	5	8	9	10	17	17	20	24	30



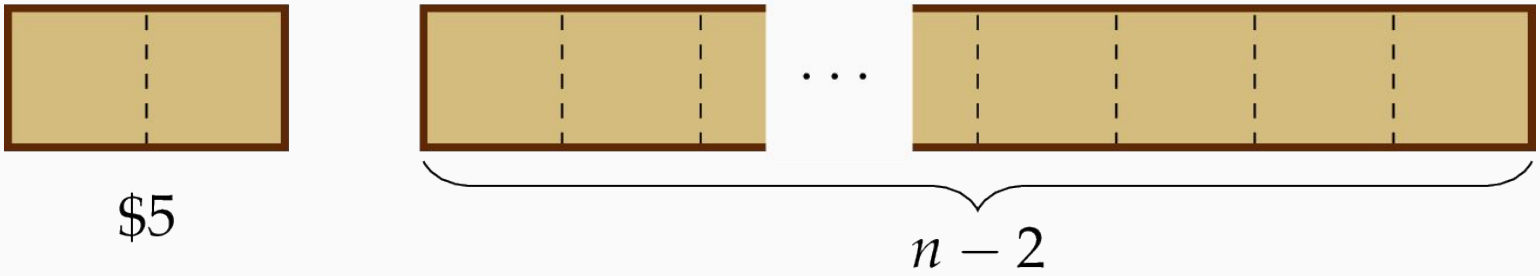
Wood cutting

length	1	2	3	4	5	6	7	8	9	10
price	1	5	8	9	10	17	17	20	24	30



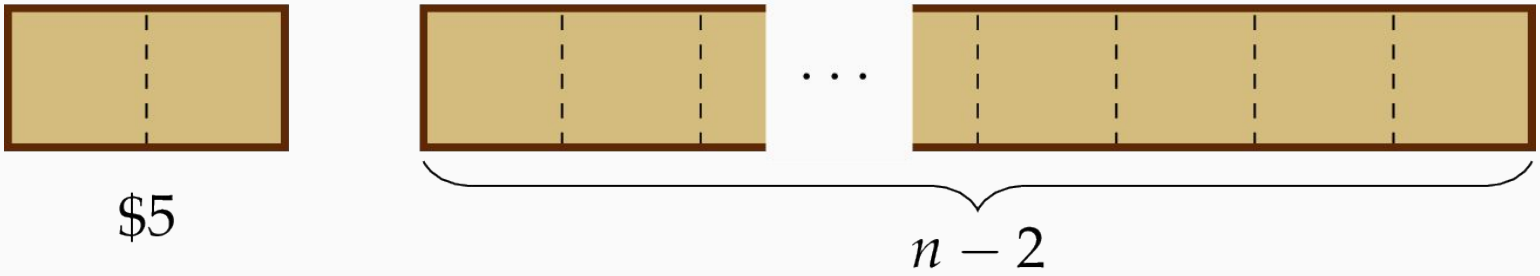


Wood cutting



How do we compute the best profit for what's left over?

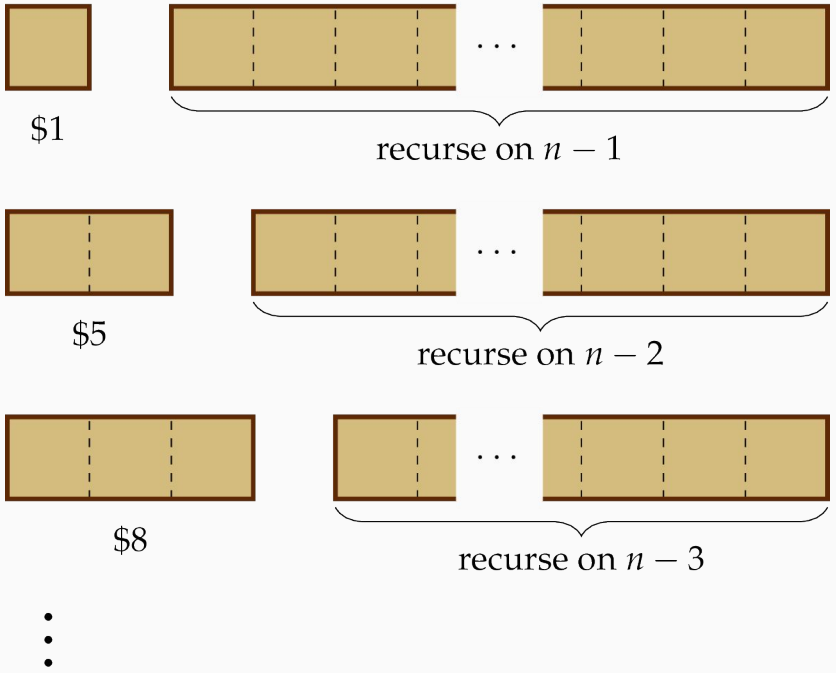
Wood cutting



How do we compute the best profit for what's left over? **Recursion!**

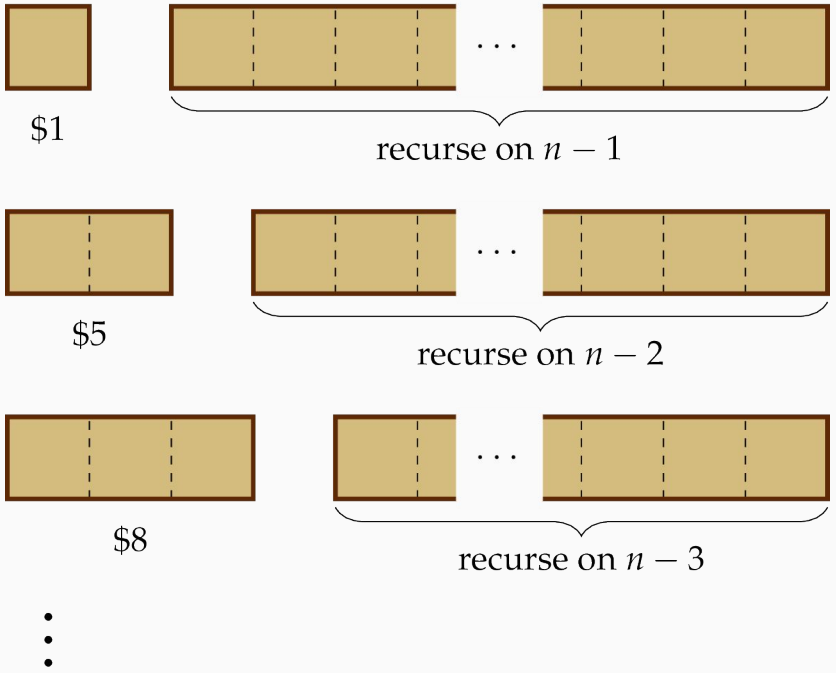
Finding the best solution

Try out all possible “first pieces” and pick the one that maximizes profit.



Finding the best solution

Try out all possible "first pieces" and pick the one that maximizes profit.



$$P(m) = \begin{cases} 0 & \text{if } m = 0 \\ \max\left(p[1] + P(m-1), p[2] + P(m-2), \ldots, p[m] + P(0)\right) & \text{otherwise} \end{cases}$$

Wood cutting

```
WoodCut(p[1,...,N], m):
    best := 0
    for i = 1,...,m
        profit := p[i] + WoodCut(p, m − i)
        if profit > best
            best := profit
    return best
```

$$P(m) = \begin{cases} 0 & \text{if } m = 0 \\ \max\big(p[1] + P(m-1), p[2] + P(m-2), \ldots, p[m] + P(0)\big) & \text{otherwise} \end{cases}$$

Wood cutting

$$
\begin{aligned}
&\textsc{WoodCut}(p[1,\ldots,N], m):\\
&\qquad \mathit{best} := 0\\
&\qquad \textbf{for } i = 1,\ldots,m\\
&\qquad\qquad \mathit{profit} := p[i] + \textsc{WoodCut}(p, m-i)\\
&\qquad\qquad \textbf{if } \mathit{profit} > \mathit{best}\\
&\qquad\qquad\qquad \mathit{best} := \mathit{profit}\\
&\qquad \textbf{return } \mathit{best}
\end{aligned}
$$

The base case should be ***m*** = 0, but where is it?

Text segmentation

Given a string of characters, can we insert spaces so that we get a sequence of words?

```
THISISTHEREASON
```

Text segmentation

Given a string of characters, can we insert spaces so that we get a sequence of words?

```
THISISTHEREASON → THIS IS THE REASON
```

Text segmentation

Given a string of characters, can we insert spaces so that we get a sequence of words?

```
THISISTHEREASON → THIS IS THE REASON

THISISTHEREASON → THIS IS THERE A SON
```

Text segmentation

Possible that some words don’t appear in any valid partition.

TH|**IS**|ISTHEREASON

Text segmentation

Possible that some words don’t appear in any valid partition.

TH | **IS** | ISTHEREASON

No greedy approaches will always work.

Text segmentation

Possible that some words don't appear in any valid partition.

`TH|IS|ISTHEREASON`

No greedy approaches will always work. **Let's solve using backtracking**.

Text segmentation

Sequence of choices: places to add spaces

Text segmentation

Sequence of choices: places to add spaces

Base case: length 0, splittable into 0 words

Text segmentation

Sequence of choices: places to add spaces

Base case: length 0, splittable into 0 words

First choice/nibble: pick out the first word. What is that first word?

Text segmentation

Sequence of choices: places to add spaces

Base case: length 0, splittable into 0 words

First choice/nibble: pick out the first word. What is that first word?

> **Try every possible prefix and see which one leads to a splitting.**

Text segmentation

First word, recurse on rest:

```
T HISISTHEREASON

TH ISISTHEREASON

THI SISTHEREASON

…
```

Text segmentation

```
SPLITTABLE(S[1,...,n]):
    if n = 0
        return TRUE
    for i = 1,...,n
        if S[1,...,i] is a word
            if SPLITTABLE(S[i+1,...,n])
                return TRUE
    return FALSE
```

Text segmentation

BOTHEARTHANDSATURNSPIN

- ~~B OTHEARTHANDSATURNSPIN~~
- ~~BO THEARTHANDSATURNSPIN~~
- BOT HEARTHANDSATURNSPIN
- BOTH EARTHANDSATURNSPIN
- ~~BOTHE ARTHANDSATURNSPIN~~

Text segmentation

What’s the worst-case scenario?

Text segmentation

What’s the worst-case scenario? **An almost-splittable string**.

Text segmentation

What’s the worst-case scenario? **An almost-splittable string**.

- Dictionary: a, aa
- String: aaaaaaaaaaaaaaaa...aaaaaaaaab

Text segmentation

What's the worst-case scenario? **An almost-splittable string**.

- Dictionary: a, aa
- String: aaaaaaaaaaaaaaaa...aaaaaaaaab

$$T(n) = T(n-1) + T(n-2) + O(1) = \Theta(\phi^n)$$

$$\phi = \frac{1+\sqrt{5}}{2} \approx 1.618\ldots$$